

DDA5001 Machine Learning

Gradient-based Optimization Algorithm

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



Recap: Convex Instances in Machine Learning

We have the following functions are convex:

- ▶ **Least squares:**

$$\mathcal{L}(\boldsymbol{\theta}) = \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2.$$

Convex because the Hessian $\nabla^2 \mathcal{L}(\boldsymbol{\theta}) = 2\mathbf{X}^\top \mathbf{X}$ is PSD.

- ▶ **Logistic regression:**

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \log \left(1 + \exp \left(-y_i \cdot \boldsymbol{\theta}^\top \mathbf{x}_i \right) \right).$$

- ▶ **Multi-class logistic regression:**

$$\mathcal{L}(\boldsymbol{\Theta}) = -\frac{1}{n} \sum_{i=1}^n \sum_{\ell=1}^K 1_{\{y_i=\ell\}} \log \left(\frac{\exp(\boldsymbol{\theta}_\ell^\top \mathbf{x}_i)}{\sum_{j=1}^K \exp(\boldsymbol{\theta}_j^\top \mathbf{x}_i)} \right).$$

- ▶ SVM learning problem (later).

Recap: Gradient Descent

Gradient descent (GD)

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

- ▶ μ_k is the **learning rate** / **stepsize**.

GD is equivalent to

$$\boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} l_k(\boldsymbol{\theta}) := \mathcal{L}(\boldsymbol{\theta}_k) + \nabla \mathcal{L}(\boldsymbol{\theta}_k)^\top (\boldsymbol{\theta} - \boldsymbol{\theta}_k) + \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2$$

- ▶ $\mathcal{L}(\boldsymbol{\theta}_k) + \nabla \mathcal{L}(\boldsymbol{\theta}_k)^\top (\boldsymbol{\theta} - \boldsymbol{\theta}_k)$ is **linear approximation** of $\mathcal{L}$ at $\boldsymbol{\theta}_k$.
- ▶ $\frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2$ is **proximal term** related to learning rate μ_k .
- ▶ $l_k(\boldsymbol{\theta})$ is a quadratic function to be minimized at each iteration.

Recap: A Useful Algorithm Design Framework

Suppose the task is $\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta})$, we can design an algorithm as

$$\boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \left\{ q_k(\boldsymbol{\theta}) + \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2 \right\}$$

μ_k is learning rate-like quantity.

- ▶ When $q_k(\boldsymbol{\theta})$ is **linear approximation** of $\mathcal{L} \implies$ **gradient descent**
- ▶ When $q_k(\boldsymbol{\theta})$ is **second-order approximation** of $\mathcal{L} \implies$ **Newton's method**
- ▶ When $q_k(\boldsymbol{\theta})$ is $\mathcal{L}$ itself $\implies$ **proximal point method**
- ▶ When $q_k(\boldsymbol{\theta})$ is **single component linear approximation** of $\mathcal{L} \implies$ **stochastic gradient descent (SGD)**
- ▶ ...

Many optimization algorithms follow this designing framework.

Recap: Convergence of GD

- Suppose that $\mathcal{L}$ is **convex** and **differentiable** and has **Lipschitz continuous gradient** with parameter L ,

$$\|\nabla\mathcal{L}(\mathbf{w}) - \nabla\mathcal{L}(\mathbf{u})\|_2 \leq L\|\mathbf{w} - \mathbf{u}\|_2, \quad \forall \mathbf{w}, \mathbf{u}$$

↪ Both convexity and Lipschitz gradient are satisfied in LR.

Theorem: Convergence and Convergence rate of GD

Gradient descent with constant learning rate $\mu_k = \mu = 1/L$ satisfies

$$\mathcal{L}(\boldsymbol{\theta}_k) - \mathcal{L}(\hat{\boldsymbol{\theta}}) \leq \frac{L\|\boldsymbol{\theta}_0 - \hat{\boldsymbol{\theta}}\|_2^2}{2k}$$

- $\mathcal{L}(\boldsymbol{\theta}_k)$ converges to $\mathcal{L}(\hat{\boldsymbol{\theta}})$ at the rate of $\mathcal{O}(1/k)$.

Learning Rate, Stopping Criterion, and Applying GD to LR

The Choice of Learning Rate

Gradient descent:

$$\theta_{k+1} = \theta_k - \mu_k \nabla \mathcal{L}(\theta_k)$$

- ▶ In practice, the simplest choice for the learning rate is to use a **constant learning rate**, i.e., $\mu_k = \mu$ for $k = 0, 1, \dots$ for some relatively small μ . Some typical guess: 0.05, 0.01, 0.005...
 $\leadsto$ In machine learning, a **larger** model often needs a **smaller** constant learning rate.
- ▶ In optimization, one can also use certain kind of **line-search** method for choosing μ_k in an adaptive manner.
 $\leadsto$ However, in machine learning, line-search is not used as it wastes too many computations.
- ▶ We will introduce **decaying learning rate** schedules when we study stochastic gradient descent.

Stopping Criterion for GD

A practical question: Run GD, when to stop?

- ▶ We cannot let k go to infinity in practice since the algorithm will never stop in this way.
- ▶ We need a practical **stopping criterion**.

Stopping Criterion for GD

A practical question: Run GD, when to stop?

- ▶ We cannot let k go to infinity in practice since the algorithm will never stop in this way.
- ▶ We need a practical **stopping criterion**.

Typical stopping criteria:

- ▶ Fix the total number of iterations as K .

Stopping Criterion for GD

A practical question: Run GD, when to stop?

- ▶ We cannot let k go to infinity in practice since the algorithm will never stop in this way.
- ▶ We need a practical **stopping criterion**.

Typical stopping criteria:

- ▶ Fix the total number of iterations as K .
- ▶ Stop when $\|\nabla\mathcal{L}(\boldsymbol{\theta}_k)\|_2$ is small, say $\|\nabla\mathcal{L}(\boldsymbol{\theta}_k)\|_2 \leq \varepsilon$.

Stopping Criterion for GD

A practical question: Run GD, when to stop?

- ▶ We cannot let k go to infinity in practice since the algorithm will never stop in this way.
- ▶ We need a practical **stopping criterion**.

Typical stopping criteria:

- ▶ Fix the total number of iterations as K .
- ▶ Stop when $\|\nabla\mathcal{L}(\boldsymbol{\theta}_k)\|_2$ is small, say $\|\nabla\mathcal{L}(\boldsymbol{\theta}_k)\|_2 \leq \varepsilon$.
 - since $\|\nabla\mathcal{L}(\hat{\boldsymbol{\theta}})\|_2 = 0$ at minimum $\hat{\boldsymbol{\theta}}$.
- ▶ Stop when $\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k\|_2$ is small, say $\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k\|_2 \leq \varepsilon$.

Stopping Criterion for GD

A practical question: Run GD, when to stop?

- ▶ We cannot let k go to infinity in practice since the algorithm will never stop in this way.
- ▶ We need a practical **stopping criterion**.

Typical stopping criteria:

- ▶ Fix the total number of iterations as K .
- ▶ Stop when $\|\nabla \mathcal{L}(\boldsymbol{\theta}_k)\|_2$ is small, say $\|\nabla \mathcal{L}(\boldsymbol{\theta}_k)\|_2 \leq \varepsilon$.
 - since $\|\nabla \mathcal{L}(\hat{\boldsymbol{\theta}})\|_2 = 0$ at minimum $\hat{\boldsymbol{\theta}}$.
- ▶ Stop when $\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k\|_2$ is small, say $\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k\|_2 \leq \varepsilon$.
- ▶ **In machine learning**, stop when **validation error** is going to increase.
($\rightsquigarrow$ Later)

Applying GD to LR

Given a set of training data points: $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, the learning problem of binary LR is:

$$\hat{\boldsymbol{\theta}} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \left\{ \mathcal{L}(\boldsymbol{\theta}) := \frac{1}{n} \sum_{i=1}^n \log \left(1 + \exp \left(-y_i \cdot \boldsymbol{\theta}^\top \mathbf{x}_i \right) \right) \right\}.$$

- ▶ $\mathcal{L}$ is convex and has Lipschitz gradient.
- ▶ We can apply GD:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

and it has convergence guarantee of $\mathcal{L}(\boldsymbol{\theta}_k) - \mathcal{L}(\hat{\boldsymbol{\theta}}) \leq \mathcal{O}(1/k)$ if the learning rate is chosen properly.

- ▶ The main elements are: 1. Compute the gradient and form the search direction. 2. Determine learning rate (usually a small constant). 3. Stop when the stopping criterion is satisfied.
- ▶ Need to know how to compute the gradient by chain rule.

Some Pros and Cons for GD

Pros:

- ▶ GD has simple implementation.
- ▶ It works well for almost all differentiable convex problems.

Cons:

- ▶ The convergence speed of GD is relatively slow.

Is there an method that simultaneously has simple implementation and fast convergence speed? $\rightsquigarrow$ **Acceleration technique.**

Agenda

Gradient Descent with Acceleration

Gradient Descent with Momentum

Gradient Descent with Momentum

GD:

$$\theta_{k+1} = \theta_k - \mu_k \nabla \mathcal{L}(\theta_k)$$

- ▶ GD forces sufficient decrease at each iteration, leaving the possibility of exploring more efficient search direction.

A quite popular technique for accelerating gradient descent method is the **momentum** technique:

$$\theta_{k+1} = \theta_k - \mu_k \nabla \mathcal{L}(\theta_k) + \beta_k \underbrace{(\theta_k - \theta_{k-1})}_{\text{momentum}}$$

An equivalent form:

$$\begin{aligned}\theta_{k+1} &= \theta_k - m_k \\ m_k &= \mu_k \nabla \mathcal{L}(\theta_k) + \beta_k m_{k-1}\end{aligned}$$

Gradient Descent with Momentum

GD:

$$\theta_{k+1} = \theta_k - \mu_k \nabla \mathcal{L}(\theta_k)$$

- ▶ GD forces sufficient decrease at each iteration, leaving the possibility of exploring more efficient search direction.

A quite popular technique for accelerating gradient descent method is the **momentum** technique:

$$\theta_{k+1} = \theta_k - \mu_k \nabla \mathcal{L}(\theta_k) + \beta_k \underbrace{(\theta_k - \theta_{k-1})}_{\text{momentum}}$$

An equivalent form:

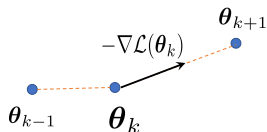
$$\begin{aligned}\theta_{k+1} &= \theta_k - m_k \\ m_k &= \mu_k \nabla \mathcal{L}(\theta_k) + \beta_k m_{k-1}\end{aligned}$$

- ▶ Due to Boris T. Polyak.
- ▶ Each iteration takes nearly the same time cost as GD.
- ▶ Widely used in practice, notably in **Adam** algorithm.

Momentum Acceleration: Geometric Interpretation

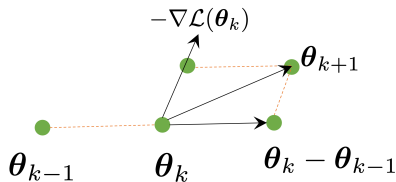
GD:

$$\theta_{k+1} = \theta_k - \mu_k \nabla \mathcal{L}(\theta_k)$$



GD with momentum:

$$\theta_{k+1} = \theta_k - \alpha_k \nabla \mathcal{L}(\theta_k) + \beta_k (\theta_k - \theta_{k-1})$$



Nesterov's Accelerated Gradient Descent

Nesterov's Acceleration

GD:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

Another very useful technique to accelerate GD is **Nesterov's accelerated gradient descent (AGD)**:

$$\begin{aligned}\boldsymbol{\theta}_{k+1} &= \boldsymbol{w}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{w}_k) \\ \boldsymbol{w}_k &= \boldsymbol{\theta}_k + \frac{k-1}{k+2}(\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1})\end{aligned}$$

Nesterov's Acceleration

GD:

$$\theta_{k+1} = \theta_k - \mu_k \nabla \mathcal{L}(\theta_k)$$

Another very useful technique to accelerate GD is **Nesterov's accelerated gradient descent (AGD)**:

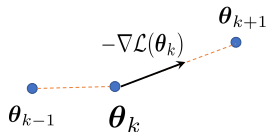
$$\begin{aligned}\theta_{k+1} &= w_k - \mu_k \nabla \mathcal{L}(w_k) \\ w_k &= \theta_k + \frac{k-1}{k+2}(\theta_k - \theta_{k-1})\end{aligned}$$

- ▶ Name due to Yurii Nesterov.
- ▶ Nesterov's acceleration is motivated by momentum but with gradient evaluated at the extrapolated point and a specific choice of β_k .
- ▶ Each iteration takes nearly the same time cost as GD.
- ▶ Widely used in practice.

Nesterov's Acceleration: Geometric Interpretation

GD:

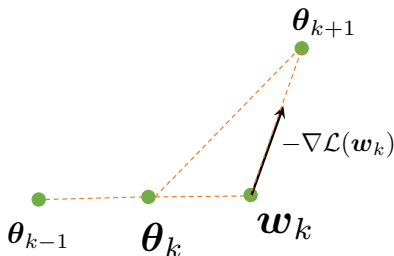
$$\theta_{k+1} = \theta_k - \mu_k \nabla \mathcal{L}(\theta_k)$$



AGD:

$$\theta_{k+1} = w_k - \mu_k \nabla \mathcal{L}(w_k)$$

$$w_k = \theta_k + \frac{k-1}{k+2}(\theta_k - \theta_{k-1})$$



Convergence of AGD

- ▶ Suppose that $\mathcal{L}$ is convex and differentiable and has **Lipschitz continuous gradient** with parameter L (For example, LR problem).

Theorem: Convergence of AGD

Accelerated gradient descent with constant learning rate $\mu_k = \mu = 1/L$ satisfies

$$\mathcal{L}(\boldsymbol{\theta}_k) - \mathcal{L}(\hat{\boldsymbol{\theta}}) \leq \frac{2L\|\boldsymbol{\theta}_0 - \hat{\boldsymbol{\theta}}\|_2^2}{(k+1)^2}$$

- ▶ $\mathcal{L}(\boldsymbol{\theta}_k)$ converges to $\mathcal{L}(\hat{\boldsymbol{\theta}})$ at the rate of $\mathcal{O}(1/k^2)$.
- ▶ This rate is much faster than GD, which only has $\mathcal{O}(1/k)$ convergence rate.

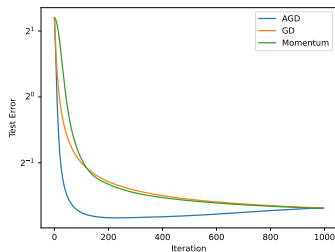
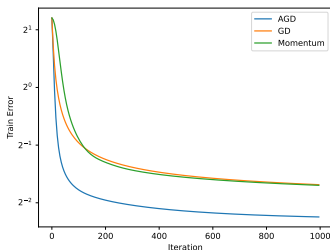
Solving Logistic Regression

Learning problem: Multi-class logistic regression

$$\hat{\Theta} = \underset{\Theta \in \mathbb{R}^{(d+1) \times K}}{\operatorname{argmin}} -\frac{1}{n} \sum_{i=1}^n \sum_{\ell=1}^K 1_{\{y_i=\ell\}} \log \left(\frac{\exp(\theta_{\ell}^{\top} \mathbf{x}_i)}{\sum_{j=1}^K \exp(\theta_j^{\top} \mathbf{x}_i)} \right)$$

Setup:

- ▶ **MNIST** classification. $n = 60000, d = 784, K = 10$.
- ▶ $\beta_k = 0.98$ in GD with momentum.
- ▶ Learning rate $\mu_k = 2 \times 10^{-2}$.



⇒ We will implement in HW2.

- ▶ AGD is the fastest one.